Assignment 5
Implementation of the SMO Algorithm for Hard-Margin SVM

Assigned: 11/10/2026
Due on Canvas: 11/24/2026, 11:59PM

50 points (+5 bonus)

1 Overview

Write your own Python (or Matlab) implementation of the Sequential Minimal Optimization (SMO) algorithm for the *hard-margin* SVM, following the description below. Your implementation must work with an arbitrary kernel; you will use two:

$$K_1(\mathbf{x}, \mathbf{z}) = \mathbf{x} \cdot \mathbf{z} \qquad \text{and} \qquad K_2(\mathbf{x}, \mathbf{z}) = \big[K_1(\mathbf{x}, \mathbf{z}) + 1\big]^2.$$

You will run both kernels on the same data sets (Section 4), plot the separating surfaces, report confusion matrices, verify your solution, and discuss what you observe.

What you may use. NumPy/Matlab linear algebra, Matplotlib (or Matlab plotting), and pandas are fine. You may use scikit-learn *only* to load data, split train/test sets, compute confusion matrices, and for the comparison in Part C. You may **not** call any SVM or QP solver inside your SMO implementation.

2 Background: the hard-margin dual and its KKT conditions

Given a training set $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$, $y_i \in \{-1, +1\}$, and a kernel K with Gram matrix $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$, the hard-margin SVM dual problem is

$$\max_{\boldsymbol{\alpha}} \; W(\boldsymbol{\alpha}) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j K_{ij} \quad \text{subject to} \quad \alpha_i \geq 0, \;\; \sum_{i=1}^m y_i \alpha_i = 0.$$

The decision function is

$$f(\mathbf{x}) = \sum_{j=1}^m \alpha_j y_j K(\mathbf{x}_j, \mathbf{x}) + b, \qquad \text{predicted label } = \operatorname{sign} f(\mathbf{x}).$$

For the linear kernel K_1 this is the familiar $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$ with $\mathbf{w} = \sum_j \alpha_j y_j \mathbf{x}_j$. For K_2 the weight vector lives in the (implicit) feature space, so you must always evaluate f through the kernel sum.

At the optimum, every training point satisfies the KKT conditions

$$\alpha_i \geq 0, \qquad y_i f(\mathbf{x}_i) \geq 1, \qquad \alpha_i \big[y_i f(\mathbf{x}_i) - 1\big] = 0. \tag{1}$$

That is, either $\alpha_i = 0$ and the point is on or outside the margin, or $\alpha_i > 0$ and the point lies exactly on the margin (a *support vector*).

3 The Hard-Margin SMO Algorithm

Input: a training set S that is separable in the feature space of K; a kernel K; a threshold $\epsilon > 0$ for zeroing multipliers (e.g. 10^{-8}); a KKT tolerance $\tau > 0$ (e.g. 10^{-3}); a maximum number of iterations T (e.g. 10^4).

1. Initialize. Pre-compute the Gram matrix K_{ij}. Initialize $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_m)$ *randomly*, with $\alpha_i \geq 0$ and

$$\sum_{i=1}^{m} y_i \alpha_i = 0.$$

One simple way: draw $u_i \sim U(0,1)$, then set $\alpha_i = u_i / \sum_{j:\, y_j = +1} u_j$ for positive examples and $\alpha_i = u_i / \sum_{j:\, y_j = -1} u_j$ for negative examples. Set $b = 0$. Record the random seed you use.

2. Evaluate the decision function on all training points: $f(\mathbf{x}_i) = \sum_{j=1}^{m} \alpha_j y_j K_{ji} + b$, for $i = 1, \ldots, m$. (For K_1 you may equivalently compute $\mathbf{w} = \sum_j \alpha_j y_j \mathbf{x}_j$ and $f(\mathbf{x}_i) = \mathbf{w} \cdot \mathbf{x}_i + b$.)

3. Compute errors and KKT quantities. For $i = 1, \ldots, m$:

$$E(i) = f(\mathbf{x}_i) - y_i = \sum_{j=1}^{m} \alpha_j y_j K_{ji} + b - y_i,$$

$$KKT(i) = \alpha_i \big[y_i f(\mathbf{x}_i) - 1 \big],$$

$$v(i) = \begin{cases} \big|\, y_i f(\mathbf{x}_i) - 1 \,\big| & \text{if } \alpha_i > 0, \\ \max\big(0,\ 1 - y_i f(\mathbf{x}_i)\big) & \text{if } \alpha_i = 0. \end{cases}$$

$v(i)$ measures how badly point i violates (1). (Selecting by $\arg\max_i KKT(i)$ alone would miss points with $\alpha_i = 0$ that lie inside the margin, since $KKT(i) = 0$ for them; $v(i)$ catches both kinds of violation.) **Stop** if $\max_i v(i) < \tau$; the current $\boldsymbol{\alpha}, b$ is the solution.

4. Pick the pair (i_1, i_2).

(a) $i_1 = \arg\max_i v(i)$ (the worst KKT violator).

(b) Compute $e(i) = E(i_1) - E(i) = \sum_{j=1}^{m} \alpha_j y_j (K_{ji_1} - K_{ji}) + y_i - y_{i_1}$, for $i = 1, \ldots, m$.

(c) $i_2 = \arg\max_{i \neq i_1} |e(i)|$, considering only i for which $k = K_{i_1 i_1} + K_{ii} - 2K_{i_1 i} > 0$.

(d) Set $k = K_{i_1 i_1} + K_{i_2 i_2} - 2K_{i_1 i_2}$.

5. Update α_{i_2} and clip it to the feasible segment.

$$\alpha_{i_2}^{\text{unc}} = \alpha_{i_2}^{\text{old}} + \frac{y_{i_2}\, e(i_2)}{k}, \qquad \alpha_{i_2}^{\text{new}} = \min\big(\max(\alpha_{i_2}^{\text{unc}}, L),\, H\big),$$

where

$$\begin{aligned} &\text{if } y_{i_1} \neq y_{i_2}: \quad L = \max\big(0,\ \alpha_{i_2}^{\text{old}} - \alpha_{i_1}^{\text{old}}\big), \quad H = +\infty; \\ &\text{if } y_{i_1} = y_{i_2}: \quad L = 0, \qquad\qquad\qquad\quad H = \alpha_{i_1}^{\text{old}} + \alpha_{i_2}^{\text{old}}. \end{aligned}$$

The bounds guarantee that both multipliers stay non-negative after Step 6.

6. Update α_{i_1} so that $\sum_i y_i \alpha_i = 0$ is preserved:

$$\alpha_{i_1}^{\text{new}} = \alpha_{i_1}^{\text{old}} + y_{i_1} y_{i_2} \big(\alpha_{i_2}^{\text{old}} - \alpha_{i_2}^{\text{new}}\big).$$

If α_{i_2} did not change (the step was clipped to zero length), try the next-best i_2 in the order of Step 4(c); if no i_2 produces a change, try the next-worst violator as i_1. If no pair can make progress, stop and report it.

7. **Clean up.** For $i = 1, \ldots, m$: if $\alpha_i < \epsilon$, set $\alpha_i \leftarrow 0$.

8. **Recompute** b from the KKT conditions. For every support vector ($\alpha_s > 0$) we must have $y_s f(\mathbf{x}_s) = 1$, i.e. $b = y_s - \sum_j \alpha_j y_j K_{js}$. For numerical stability, average over all of them:

$$b = \frac{1}{|SV|} \sum_{s \in SV} \Big(y_s - \sum_{j=1}^{m} \alpha_j y_j K_{js} \Big), \qquad SV = \{s : \alpha_s > 0\}.$$

9. **Monitor.** Record, for this iteration: the dual objective $W(\boldsymbol{\alpha})$, the number of support vectors, the number of misclassified training points, and $\left|\sum_i y_i \alpha_i\right|$.

10. **Repeat** from Step 2 until the stopping test in Step 3 succeeds or T iterations have been performed.

3.1 Notes

- Do <u>not</u> rename i_1, i_2 as $1, 2$ in your code; keep the indices. That is, $y_1 \to y_{i_1}$, $\alpha_1 \to \alpha_{i_1}$, $K_{12} \to K_{i_1 i_2}$, $E(1) \to E(i_1)$, and so on.
- **"All training points correctly classified" is not the same as "optimal".** A random $\boldsymbol{\alpha}$ can already separate the data while giving a poor margin. That is why the stopping test is the KKT test in Step 3, not the training accuracy. Report at which iteration the training error first reached zero, and compare it with the iteration at which the KKT test was met.
- The pair update in Steps 5–6 maximizes W along the constraint line, so $W(\boldsymbol{\alpha})$ should never decrease, apart from tiny changes caused by Step 7. A decreasing W almost always means a bug, typically a sign error in $e(i)$ or in the update.
- Hard margin assumes the data are separable in the feature space of the kernel. If they are not, the algorithm cannot converge. Part D asks you to observe what happens instead.

4 Data

All data sets are posted on Canvas as CSV files (`SMO_data.zip`), so that all of you work on exactly the same points, whatever language you use.

- Each file has a header row; the last column, `y`, holds the label (± 1) and the other columns are the features.
- Use the train/test files exactly as given; do not re-split.
- Standardize features (zero mean, unit variance, using *training-set* statistics only) unless told otherwise, and say whether you did.
- Use a fixed random seed for the initialization of $\boldsymbol{\alpha}$, and report it.

Data set 1: Iris, setosa vs. versicolor. `iris_setosa_versicolor_train.csv` (70 points) and `iris_setosa_versicolor_test.csv` (30 points). Features: *sepal length* and *petal length* (cm); setosa is $+1$, versicolor -1. These two classes are linearly separable.

Data set 2: "XOR" (synthetic). `xor_train.csv` (200 points) and `xor_test.csv` (100 points). Points are uniform in $[-2, 2]^2$, with a gap of width 0.3 around both axes, and $y = \text{sign}(x_1 x_2)$. Not linearly separable in $\mathbb{R}^2$, but separable with K_2.

Data set 3: concentric rings (synthetic). `rings_train.csv` (200 points) and `rings_test.csv` (100 points). Inner disk of radius 1 (label $+1$) and a ring with radius between 1.5 and 2.5 (label -1), angles drawn uniformly; the two classes are balanced.

Data set 4: Iris, versicolor vs. virginica (for Part D). `iris_versicolor_virginica.csv` (100 points, no split needed). Features: *petal length* and *petal width*; versicolor is $+1$, virginica -1. These classes *overlap*.

Toy data. Also run your code on the toy data set supplied on Canvas, as a sanity check.

5 What to do

Part A: Implementation (20 points). Implement the algorithm above as a function `smo(X, y, kernel, eps, tau, T, seed)` that returns $\boldsymbol{\alpha}$, b, and the per-iteration history from Step 9. Kernels must be passed in as functions, so that adding a new kernel requires no change to `smo`. Document your code.

Part B: Experiments (12 points). For each of data sets 1–3, and for each kernel K_1 and K_2:

1. Plot the training data, the decision boundary $f(\mathbf{x}) = 0$, the margins $f(\mathbf{x}) = \pm 1$, and circle the support vectors. (Evaluate f on a grid and use a contour plot. This works for both kernels.)
2. Give the confusion matrix on the test set (and the training set).
3. Fill in one row of a summary table:

Data	Kernel	Converged?	Iterations	#SV	b	Margin $2/\|\mathbf{w}\|$	Test acc.
Iris 1	K_1						
...							

   For any kernel, $\|\mathbf{w}\|^2 = \sum_{i,j} \alpha_i \alpha_j y_i y_j K_{ij}$; at the optimum this also equals $\sum_i \alpha_i$. Check that the two agree.
4. Plot $W(\boldsymbol{\alpha})$ and the number of misclassified training points versus iteration.

Part C: Verification (6 points).

1. Run SMO on data set 1 with K_1 from 10 different random initializations. Is the final hyperplane $(\mathbf{w}, b)$ the same each time? Are the α_i the same? Explain.
2. Compare your result on data set 1 (both kernels) with `sklearn.svm.SVC(kernel='linear', C=1e8)` and `SVC(kernel='poly', degree=2, gamma=1, coef0=1, C=1e8)`, which approximate the hard-margin SVM. Compare b, the support vectors, and $W(\boldsymbol{\alpha})$. They should agree to several decimal places.
3. Confirm that $\sum_i y_i \alpha_i = 0$ (to round-off) and that every $\alpha_i \geq 0$ throughout the run.

Part D: When hard margin fails (4 points).

1. Run SMO on data set 2 with K_1 and on data set 4 with both kernels.
2. Plot $\max_i \alpha_i$ and $W(\boldsymbol{\alpha})$ versus iteration.

(a) What happens, and why?

(b) Relate your answer to the dual: what does it mean for W to be unbounded?

3. Explain in a few sentences how the soft-margin SVM (the constraint $0 \le \alpha_i \le C$) fixes this.

Part E: Report and discussion (8 points). A PDF report of at most 6 pages (plus appendix plots, if needed) containing all plots, tables and confusion matrices from Parts B–D, and brief answers to:

1. On Iris (data set 1), does K_2 help? Compare the margin, the number of support vectors, and the shape of the boundary between the two kernels.
2. Why does K_1 fail on XOR and on the rings, while K_2 succeeds? For $\mathbf{x} \in \mathbb{R}^2$, write out an explicit map $\phi : \mathbb{R}^2 \to \mathbb{R}^6$ with $K_2(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x}) \cdot \phi(\mathbf{z})$, and identify which coordinate(s) of ϕ make each data set separable.
3. How many iterations did SMO need, compared with the number of training points? Why are so many iterations spent driving the initial random α_i to zero? (Hint: how many α_i are nonzero at the start, and at the end?)
4. Did standardizing the features change the number of iterations, the support vectors, or the decision boundary? Why might it matter for K_2 in particular?

Bonus (up to 5 points). Choose one:

- **Real data, honestly.** Apply your SMO to two species from the Palmer Penguins data set (`https://allisonhorst.github.io/palmerpenguins/`) using two morphological features of your choice. First determine whether the two classes are separable with K_1, with K_2, or neither, and justify how you know. Then report results as in Part B.
- **Higher degree.** Generalize K_2 to $K_d(\mathbf{x}, \mathbf{z}) = (\mathbf{x} \cdot \mathbf{z} + 1)^d$. For $\mathbf{x} \in \mathbb{R}^n$, give the dimension of the implicit feature space as a function of n and d, and show how the boundary on data set 3 changes for $d = 2, 3, 4$.
- **Better pair selection.** Implement a different rule for choosing i_1 or i_2 (for example, Platt's second-choice heuristic) and compare the number of iterations needed.

6 Submission

Upload to Canvas: (1) your source code (a single `.py`/`.ipynb` or `.m` file, or a zip with a short README explaining how to run it), and (2) your report as a PDF. Your code must reproduce every figure in the report when run with the seeds you report.

7 Grading summary

Part A: Implementation (correct, general in the kernel, documented)	20
Part B: Experiments (plots, confusion matrices, table, W curves)	12
Part C: Verification (random restarts, comparison with `SVC`, constraints)	6
Part D: Non-separable cases	4
Part E: Report and discussion	8
Total	50
Bonus	+5